

Gerenciamento de Sessão e Controle de Cache

Objetivo da Aula

Compreender o uso de sessão e cache nos sistemas web, além de conhecer exemplos de código para o gerenciamento desses recursos.

Apresentação

Algo que pode modificar completamente a forma de projetar um sistema é a possibilidade de armazenar estados de um objeto ou não. Elementos que são capazes de manter um estado são chamados de **stateful**, enquanto aqueles que não conseguem são classificados como **stateless**.

O protocolo HTTP não tem a característica de armazenamento de estados, pois cada requisição é completamente independente da anterior. Mas, como não são raras as situações em que precisamos manter o estado, seja para efetuar uma transação mais longa, com várias requisições, ou para ganhar eficiência, o uso de ferramentas auxiliares para gerenciamento de estados é necessário.

Aqui, veremos como funcionam essas ferramentas, que, no caso, são: cookies, sistemas de cache e gerenciamento de sessão.

1. Componentes Stateless e Stateful

O estado de um componente, ou de uma aplicação como um todo, é definido pelos valores que são utilizados para o conjunto de seus atributos em um dado instante. A depender do ciclo de vida do componente, esses valores podem ser mantidos por mais ou menos tempo, variando desde uma transição simples, do tipo chamada e resposta, até a persistência dos dados em meio não volátil.

Considerando a manutenção dos valores em memória, componentes do tipo **stateless** são capazes de manter os valores para os atributos apenas no escopo da transação corrente, enquanto componentes **stateful** adotam um ciclo de vida mais longo, mantendo os

valores entre chamadas sucessivas. Por exemplo, um Time Server pode ser stateless, já que a hora muda a cada nova chamada, mas uma cesta de compras virtual assume um comportamento stateful.

Indo além do uso da memória, alguns componentes podem ter ciclos de vida ainda mais longos, como a personalização de um site ou armazenagem de dados cadastrais. Nesses casos, é necessário efetuar a **persistência** dos valores em meio não volátil, como cookies e bancos de dados.



Destaque

Um componente persistente é capaz de armazenar seu estado em meio não volátil, com o objetivo de recuperá-lo posteriormente.

Um processo stateless trata de recursos isolados, em que as informações de transações antigas não são armazenadas, e cada operação é efetuada apenas no escopo da chamada. Você poderá observar o comportamento stateless em pesquisas efetuadas na web e na chamada de Web Services SOAP, já que uma interrupção no processo iria exigir que fosse reiniciado, com nova solicitação.

Já um processo stateful pode ser utilizado diversas vezes, sendo mantido o último estado, como serviços bancários e edição de documentos on-line. Se uma transação stateful for interrompida, você conseguirá continuar de onde parou, já que ocorre o armazenamento do contexto e do histórico, os quais gerenciam o estado do processo, sendo comum a utilização do mesmo servidor para a operação.

Existem diversos exemplos de componentes que adotam essa divisão entre stateless e stateful, como os Enterprise Java Beans, da plataforma Java, ou os Widgets do Flutter. Inclusive, um aplicativo React, que executa sobre o NodeJS, adota elementos do tipo state para implementar o comportamento stateful.

O protocolo HTTP é, por padrão, stateless, ou seja, cada vez que você acessa uma URL do servidor, um novo contexto é gerenciado, eliminando dados que tenham sido utilizados em chamadas anteriores. Embora não ocorra a perda da conexão, cada requisição efetuada tem seu próprio escopo, permitindo que o comportamento seja definido como stateless.

Da mesma forma, Web Services do tipo SOAP são componentes do tipo stateless, já que cada chamada para um dado serviço ocorre de forma totalmente independente de chamadas anteriores. Conceitualmente, **Web Services RESTful são stateless**, já que a

inclusão de uma entidade via método POST irá impactar a consulta posterior efetuada pelo método GET, embora as requisições em si sejam totalmente independentes por utilizarem o protocolo HTTP.



Você Sabia?

As APIs de REST não possuem estado definido, o que significa que cada solicitação precisa incluir todas as informações necessárias para processá-lo. Em outras palavras, as APIs de REST não requerem nenhuma sessão do lado do servidor. Os aplicativos do servidor não têm permissão para armazenar nenhum dado relacionado a uma solicitação de cliente.

2. Gerenciamento de Sessões do HTTP

Já que o protocolo HTTP é stateless, torna-se necessário armazenar o estado em uma das extremidades, cliente ou servidor, para processos do tipo stateful, como no caso da autenticação de usuário. As formas mais comuns para manter as informações necessárias são o uso de cookies e de sessões no servidor.

Para armazenar informações do lado cliente, você poderá utilizar cookies, que são pequenos arquivos para guardar informações do usuário. Além de um limite de 4 Kb de armazenamento, os cookies podem ser configurados para expirar em determinado prazo.



Destaque

Os cookies são muito utilizados para guardar as preferências do usuário para determinado site, como tema ou idioma.

Nunca guarde informações críticas em cookies, pois são apenas arquivos de texto sem criptografia, não apresentando mecanismos de segurança. Inclusive, os cookies podem ser apagados, eliminando as configurações armazenadas.



Você Sabia?

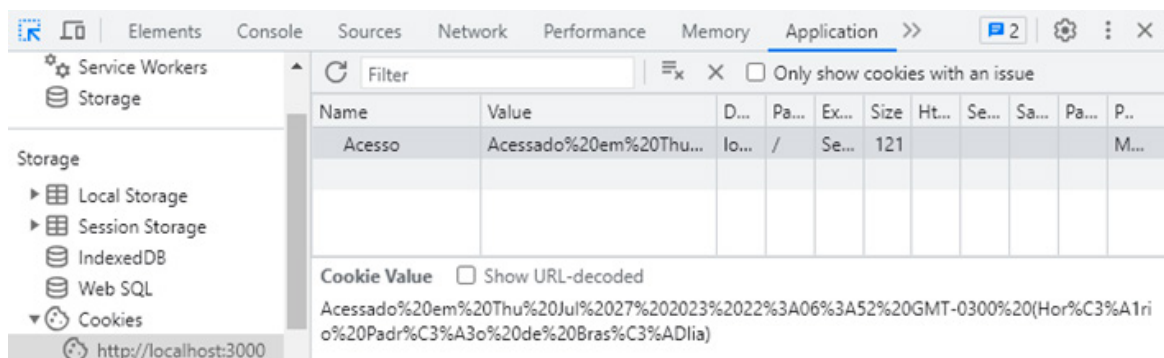
Diversas ferramentas maliciosas baseadas em scripts podem utilizar cookies, definindo uma fragilidade em termos de segurança.

O exemplo da listagem seguinte demonstra a gravação de um cookie a partir de um servidor **NodeJS**, com base na biblioteca **express**.

```
const express = require('express')
const app = express()
app.get('/setcookie', (req, res) => {
  res.cookie('Acesso', `Acessado em ${new Date()}`);
  res.send('Cookie foi salvo com sucesso');
});
app.listen(3000);
```

Após executar o servidor e acessar **http://localhost:3000/setcookie**, você pode verificar a criação do cookie, pela ferramenta de inspeção do navegador, na aba **application**, opção **cookies**, como pode ser observado a seguir.

Figura 1: Verificação do cookie no inspetor



Fonte: Elaboração própria.

Você pode ler o cookie em chamadas subsequentes ao servidor, como no exemplo seguinte, devendo ser adicionada a biblioteca **cookie-parser**.

```
const express = require('express')
const parser = require('cookie-parser')
const app = express()
app.use(parser())
app.get('/getcookie', (req, res) => {
  console.log(req.cookies);
  res.send(req.cookies['Acesso']);
});
app.listen(3000);
```

Se a necessidade do sistema é a guarda de informações, durante a conexão, como na verificação de usuário autenticado, você poderá utilizar uma sessão no servidor. Ao contrário do cookie, que trabalha no lado do cliente, para as sessões teremos as informações armazenadas no lado do servidor.

Existem outras diferenças, como o tamanho do arquivo, que pode ser muito maior, chegando a 128 Mb, e o tempo de vida, já que os dados são eliminados quando a conexão é finalizada ou a sessão invalidada. É importante observar que a segurança das sessões é muito maior que a dos cookies, embora tenha um ciclo de vida bem mais curto.

O código seguinte utiliza sessões com o express, devendo ser acrescentado o pacote **express-session**.

```
const express = require('express')
const session = require('express-session')
const app = express()
app.use(session({
  secret: 'Ax0$%s14@FX2az1k^2nM',
  resave: true,
  saveUninitialized: true
```

```

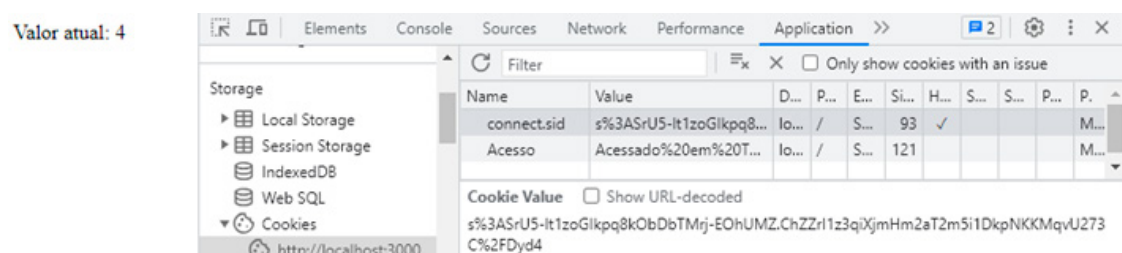
    )))
    app.get('/contador', (req, res) => {
    let contagem = 0;
    if(req.session.contagem)
    contagem = req.session.contagem;
    contagem++;
    req.session.contagem = contagem;
    res.send(`Valor atual: ${contagem}`);
    })
    app.listen(3000);

```

Você poderá acessar **http://localhost:3000/contador** e observar como a contagem aumenta a cada atualização com F5. Note que o valor é armazenado em **req.session.contagem**, sendo recuperado da mesma forma a cada nova requisição efetuada para o endereço.

Na figura seguinte, podemos observar a execução no navegador, bem como a presença de um novo cookie, que tem por objetivo identificar a sessão.

Figura 2: Uso de sessão e cookie de identificação



Fonte: Elaboração própria.

O cookie de identificação é o que permite ao servidor diferenciar as conexões, armazenando a informação correta para cada uma. Para eliminar a sessão, você pode fechar o navegador ou utilizar o comando **destroy**, ao nível do servidor.

```

app.get('/', (req, res) => {
    req.session.destroy(

```

```
(error) => {console.log("Sessao removida")});  
res.send("Sessao removida");  
})
```

3. Como Funciona o Cache

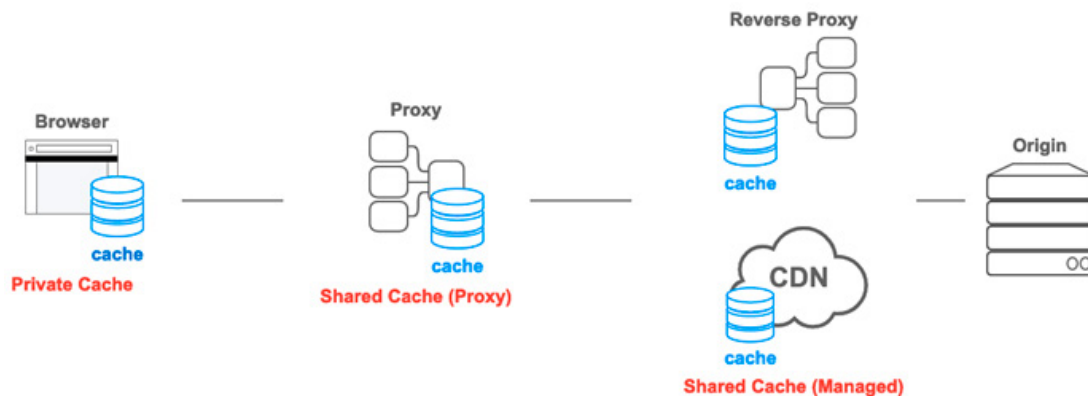
Na área de computação, um sistema de cache corresponde a qualquer meio para armazenamento de resultados recentes, buscando minimizar o tempo de acesso aos dados. A ideia não é nova e está presente em diversos componentes de hardware e software, como nas memórias cache, que são mais rápidas que as tradicionais e armazenam os valores dos últimos endereços acessados.

Em geral, os dados que serão acessados pelo processador, em um bloco de instruções, estão próximos na memória, o que leva a um ganho de eficiência, com o armazenamento em cache dos valores recentes. O mesmo pode ser aplicado às pesquisas efetuadas por determinado grupo de usuários, justificando o armazenamento de resultados recentes para ganho de eficiência.

O protocolo HTTP busca tirar o maior proveito possível dos mecanismos de cache, armazenando resultados recentes em diferentes níveis da comunicação entre o cliente e o servidor. A partir disso, ocorre a divisão em três tipos de cache, de acordo com a localização e o nível de acesso:

- **Cache privado:** é armazenado ao nível do navegador, permite guardar respostas personalizadas para determinado usuário, sem que outros usuários possam acessar;
- **Cache de proxy:** guarda respostas para as solicitações efetuadas pelos usuários internos, de forma compartilhada, visando evitar o tráfego fora da rede para solicitações equivalentes;
- **Cache gerenciado:** é implantado por desenvolvedores de serviços para diminuir o acesso ao servidor e fornecer conteúdo, de forma eficiente, a múltiplos usuários, como para as **redes de distribuição de conteúdo**, ou **CDNs**, e os **proxies reversos**.

Figura 3: Cache no protocolo HTTP



Fonte: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Caching>.



Destaque

Com a utilização cada vez mais comum de HTTPS, os sistemas de cache no proxy não conseguem guardar a informação, já que o TLS efetua a criptografia entre cliente e servidor.

Quando uma resposta HTTP está armazenada em cache, ela pode estar no estado **fresh**, no prazo em que a informação é considerada válida, ou **stale**, que indica um conteúdo expirado. Para saber o estado da informação em cache, é utilizado o atributo **age**, de forma semelhante ao time to live (**TTL**).

Você pode observar uma resposta com prazo de expiração de uma semana, ou 640800 segundos, a partir da data indicada no fragmento seguinte. Para o exemplo, a resposta seria classificada como fresh até o dia 19 de fevereiro de 2022, às 22:30:00, passando para o estado stale no segundo seguinte.

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1024
Date: Tue, 12 Feb 2022 22:30:00 GMT
Cache-Control: max-age=604800
<!doctype html>
```


Embora a indexação mais comum de uma resposta em cache seja a URL da requisição, algumas configurações podem definir conteúdo diferente no mesmo endereço, como a linguagem utilizada. Para utilizar esse tipo de classificação, o atributo **Vary** deve ser adicionado ao cabeçalho, podendo utilizar valores como Accept-Language, Accept-Encoding e User-Agent.

Quadro 1: Exemplo de uso de Vary na indexação do cache HTTP

URL	Accept-Language	Resposta
http://exemploweb1.com/index.html	ja-JP	<!doctype html>.....
http://exemploweb1.com/index.html	en-US	<!doctype html>.....

Fonte: Elaboração própria.

Da mesma forma que o protocolo HTTP tira proveito do cache, você poderá utilizar esse tipo de ferramenta para agilizar a consulta a um banco de dados ou para obter conteúdo de um servidor externo, ao criar o aplicativo NodeJS, através de gerenciadores de cache como o **redis**. Para tabelas e servidores com baixo nível de atualização, o acesso ao cache no lugar do recurso ocorrerá em tempo muito menor, trazendo grande ganho de eficiência.

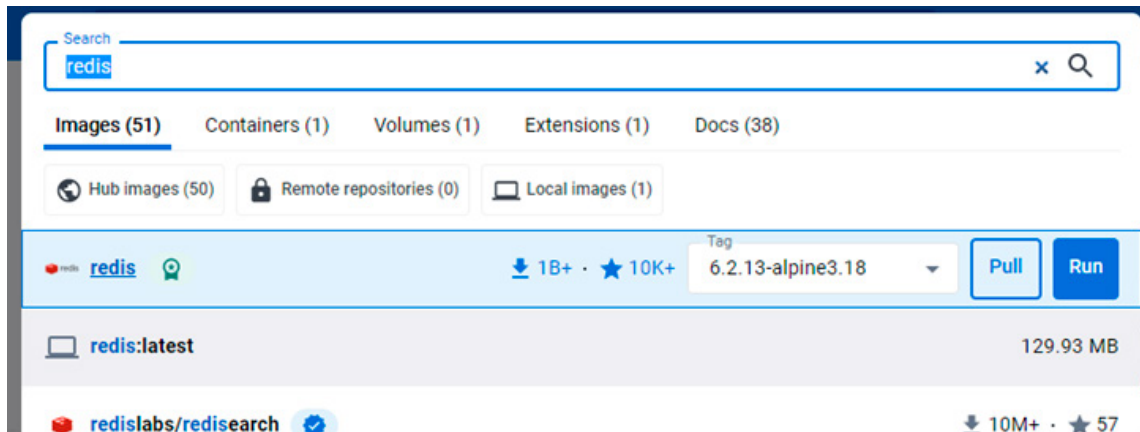


Destaque

Redis é um produto de código aberto, com armazenamento de dados em memória, e pode ser utilizado para sistemas de cache, banco de dados com acesso rápido e mecanismo de streaming.

Para simplificar o gerenciamento do ambiente, você pode executar o redis via Docker. Utilize a barra de pesquisa do **Docker Desktop**, digitando o nome do software e escolhendo a última versão (latest).

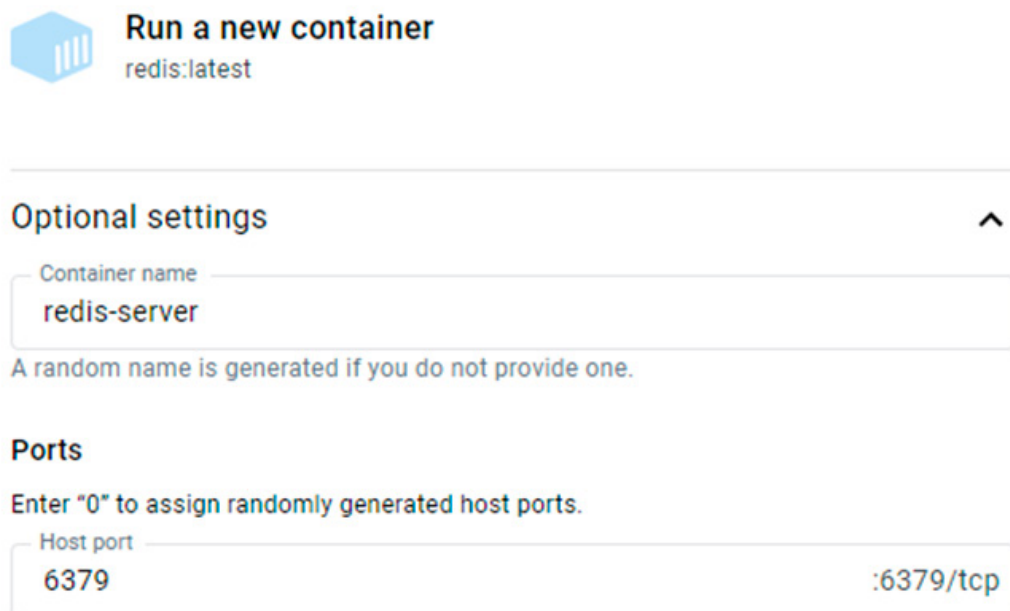
Figura 4: Acréscimo da imagem do redis



Fonte: Elaboração própria.

Após encontrar a imagem, utilize a opção **Run**, que irá baixar a imagem e criar o container. Será aberta uma tela de configuração, na qual você deve abrir o painel **optional settings**, definindo o nome do container como **redis-server** e habilitando a porta **6379**.

Figura 5: Configuração do container



Fonte: Elaboração própria.

Com todos os passos concluídos, o servidor redis estará em execução no Docker, e poderá ser utilizado a partir do sistema NodeJS, através da porta **6379** e do endereço **http://localhost**.

4. Controle de Cache

Para controlar o cache, ao nível do cliente, você deve utilizar alguns atributos no header da página. Esses atributos irão definir a forma como ocorrerá o uso do cache, tendo como base o valor de **age** para o conteúdo armazenado, bem como os estados **fresh** e **stale**.

No exemplo da listagem seguinte, será definido um cache com expiração do conteúdo após o prazo de 5 minutos (300 segundos), em uma rota no modo GET, para um servidor do tipo ExpressJS.

```
app.get('/exemplo_cache', (req, res, next) => {  
  res.set('Cache-control', 'private, max-age=300')  
  res.json({'mensagem': 'alo mundo'});  
})
```

O controle de cache do tipo **private** faz com que todas as respostas sejam armazenadas no cache do navegador. Enquanto o conteúdo não tiver expirado, uma nova consulta ao endereço irá utilizar a versão armazenada localmente.

Utilizando a configuração **public**, teremos o armazenamento da informação em qualquer cache intermediário, como nos proxies. Se a opção **no-cache** for utilizada, você obterá o mesmo tipo de comportamento, mas com a diferença de que será exigida a validação no servidor de origem para qualquer requisição.

Quando o conteúdo não for modificado para a requisição, o servidor responde com o código **304 (Not Modified)**, e o cliente acessa a resposta armazenada no cache. Caso contrário, temos uma nova resposta, com status **200**, substituindo o conteúdo atual do cache.

Uma forma prática para o servidor verificar se houve modificação é através do atributo **etag**, que pode ser um hash da resposta. Caso a resposta ainda seja a mesma, o valor de etag não será modificado.

Na listagem seguinte, temos um controle de cache para uma rota de conteúdo estático, como páginas HTML, em um servidor do tipo ExpressJS.

```
app.use(express.static('public', {
  etag: true, lastModified: true,
  setHeaders: (res, path) => {
    if (path.endsWith('.html'))
      res.setHeader('Cache-Control', 'no-cache');
    // Conteúdo em cache, mas com revalidação no servidor
  }));
```

Se desejamos um cache ao nível do servidor, precisamos utilizar o redis, já em execução no Docker. A conexão com o redis é efetuada de forma simples, com a instalação da biblioteca de mesmo nome.

```
npm install redis express
```

Com as bibliotecas adicionadas, devem ser gerados o servidor ExpressJS, que utilizará o nome **app**, e o cliente para o redis, com o nome **redisClient**.

```
const express = require('express');
const redis = require('redis');
const redisClient = redis.createClient();
redisClient.on("error", (err) => console.log(`Erro: ${err}`));
const app = express();
```

Em seguida, a resposta que será colocada em cache deverá ser obtida, com base no comando **fetch**, do JavaScript padrão. Aqui, iremos utilizar uma função que recebe a URL da requisição e retorna a resposta no formato JSON.

```
async function getUrl(url) {
  const response = await fetch(url);
  if (!response.ok) {
    throw new Error(response.statusText);
```

```
}  
return await response.json();  
}
```

O próximo passo é a definição da rota, como na listagem seguinte, em que a rota **países** fará acesso a uma URL que retorna os países do globo terrestre.

```
app.get('/paises', async (req, res) => {  
  try {  
    const statsTxt = await redisClient.get('paises');  
    if (statsTxt !== null) {  
      res.status(200).send({from: "cache",  
        content: JSON.parse(statsTxt)});  
      return;  
    }  
  } catch (err) {  
    console.log(err);  
  }  
  try {  
    const stats = await  
      getUrl('https://restcountries.com/v3.1/all');  
    redisClient.set('paises', JSON.stringify(stats), { EX: 60 });  
    res.status(200).send({from: "server", content: stats});  
  } catch (err) {  
    console.log(err);  
    res.sendStatus(500);  
  }  
});
```

Analisando o código, você pode observar que o primeiro bloco protegido tenta obter a lista de países, a partir do redis, através do comando **get**. Caso o retorno esteja no cache, ele é transmitido ao cliente, efetuando a conversão necessária por meio do método **parse**, já que no redis a lista é armazenada como texto.

Se o conteúdo não estiver no redis, o segundo bloco protegido é executado, com a chamada para **https://restcountries.com/v3.1/all**, que retorna a lista de países no formato **JSON**. Ocorrendo uma resposta positiva, o método **set** é utilizado para acrescentar o conteúdo, no formato JSON, com uma especificação de tempo para expirar em 1 minuto (ou 60 segundos).

Note que a resposta ao cliente tem dois campos, sendo que o primeiro (**from**) irá indicar se a informação veio do servidor ou do cache, e o segundo (**content**) irá trazer a lista de países.

Finalmente, tanto o cliente redis quanto o servidor ExpressJS devem ser iniciados. Como o cliente redis é utilizado no tratamento da requisição, ele deve ser iniciado antes de o servidor começar a escutar a rede.

```
redisClient.connect().then(
  app.listen(3000, console.log('Servidor na porta 3000'))
);
```

Após iniciar o aplicativo, utilize um navegador qualquer para acessar a rota especificada em <http://localhost:3000/paises>. O primeiro acesso irá indicar que a informação veio do servidor externo, e os demais indicarão a origem no cache, até que ocorra novo acesso externo, ao final do período de 60 segundos.

Figura 6: Retorno a partir do cache



Fonte: Elaboração própria.

Considerações Finais da Aula

Após aprender a diferenciar os comportamentos stateless e stateful, vimos como é possível trazer a manutenção de estados para o HTTP, que, por padrão, seria stateless, através do uso de cookies e sessões. Esses serão elementos necessários na implementação de controles de autenticação, cestas de compras e demais componentes que precisam estar disponíveis para várias requisições.

Outro elemento presente no HTTP e que visa ao ganho de performance é o cache, que pode armazenar conteúdo em vários níveis da comunicação entre cliente e servidor. Nesse aspecto, vimos como um aplicativo web pode gerenciar a utilização desse tipo de cache, com base em atributos do header HTTP.

Ainda, em termos de ganho de performance, utilizamos o redis para o cache de conteúdo ao nível do servidor. Por trabalhar com dados em memória, o redis diminui o tempo de acesso ao conteúdo armazenado, além de ser facilmente configurado no Docker e utilizado de forma direta pelo aplicativo NodeJS, desde que instalado o módulo redis no projeto.

Materiais Complementares

Artigo

HTTP caching

2023, MDN WEB DOCS.

Artigo da Mozilla que oferece informações completas acerca da utilização de cache pelo protocolo HTTP, com diversos exemplos de configurações para contextos comuns na definição de sistemas para web.

Link para acesso: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Caching> (acesso em 12 set. 2023.)

Site

Get started with Redis

2023, Redis.

Essa é a documentação oficial do Redis, na parte relacionada à instalação, ao teste e à utilização básica. Diversas informações adicionais podem ser obtidas através da navegação lateral.

Link para acesso: <https://redis.io/docs/getting-started/> (acesso em 12 set. 2023.)

Artigo

Configuring HTTP caching behavior

2018, Jeff Posnick.

O artigo oferece um Codelab com todos os passos necessários para a configuração do uso do cache HTTP em sistemas do tipo NodeJS.

Link para acesso: <https://web.dev/codelab-http-cache/> (acesso em 12 set. 2023.)

Referências

COMER, D. **Redes de computadores e internet**. Porto Alegre: Bookman, 2016. Disponível em: <https://abre.ai/gKFt>. Acesso em: 12 set. 2023.

FERREIRA, A. **Interface de programação de aplicações (API) e web services**. São Paulo: Platos Soluções Educacionais, 2021. Disponível em: <https://abre.ai/gKFv>. Acesso em: 12 set. 2023.

JERÔNIMO, A. **Práticas da cultura DevOps no desenvolvimento de sistemas**. São Paulo: Platos Soluções Educacionais, 2021. Disponível em: <https://abre.ai/gKFA>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://abre.ai/gKFB>. Acesso em: 12 set. 2023.